

Q1: The Magical Fibonacci Sequence

Background

In the land of Fibona, wizards use a special sequence to determine their powers. The first two wizards have powers of 0 and 1, respectively. Every subsequent wizard's power is determined by the sum of the powers of the two preceding wizards. You must write a recursive function to help find the power of the N-th wizard.

Input Format:

An integer N where $0 \leq N \leq 30$.

Output Format:

Return the N-th Fibonacci number.

Example

Input

5

Output

5

Q1: The Magical Fibonacci Sequence...

Explanation

0th fibo number - 0

1st fibo number - 1

2nd fibo number - 1 (0+1)

3rd fibo number - 2 (1+1)

4th fibo number - 3 (2+1)

5th fibo number - 5 (2+3)

Q2: Greatest Common Divisor

Two friends, Alex and Bob, have a tradition of sharing resources whenever they work on projects together. One day, they come across two large stacks of resources, and they want to divide them equally in a way that ensures both get the maximum amount possible without wasting any.

To do this, they need to find the largest possible size of the portions that can divide both stacks of resources evenly. They remember that they once learned an efficient way to find the largest possible size using a recursive approach called the Euclidean algorithm:

1. Find the remainder when one of the numbers is divided by the other.
2. Find the remainder when the previous divisor is divided by the previous remainder.
3. Keep repeating 2 until the remainder is zero. The final divisor that achieves this zero remainder is the required gcd.

Task:

- ❑ Write a C program that helps Alex and Bob determine the largest possible size of the two given quantities using a recursive function.
- ❑ The program should take the two resource quantities as input (two integers) and calculate the largest possible size.

Q2: Greatest Common Divisor

Input Example:

Alex's resource quantity: 48

Bob's resource quantity: 18

Output Example:

The largest possible size is: 6

(there is a newline after printing the gcd)

Input:

48 18

Output:

6

Q3: Sum of Digits

Write a C program that implements a recursive function to calculate the sum of the digits of a given integer n.

Input:

1234

Output:

10

Q1: Frobenius Inner Product

You need to provide the definition of Frobenius inner-product between two matrices. The main function has already been provided to you. Please do not make any changes in it. Only fill in the function definition.

Input Format: first two numbers denote number of rows and columns of the matrices. Next few rows are for the first matrix, subsequent rows are for the second matrix. For e.g., If the matrices are

$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ and $B = \begin{bmatrix} 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix}$, then the input will be:

2 3

1 2 3

4 5 6

7 8 9

10 11 12

Output: the printf statement has already been written for you. Please do not change it or include additional printf statements. If your function definition is correct, then the correct value of inner product between the input matrices will be printed. For your convenience, the formula for inner-product is presented below.

$$\langle A, B \rangle \equiv \sum_{i,j} A_{ij} B_{ij}$$

Q2: The Tower of Magic

Story:

In a hidden valley, there stands an ancient Tower of Magic. Legends say that this tower holds untold wisdom, but it can only be accessed by solving a mystical challenge.

The challenge is simple in concept but complex in execution. At the top of the tower lives an old sorcerer who has a set of enchanted keys, each able to unlock a certain level of the tower. To climb the tower, you must determine the number of ways to reach the top, but there's a catch: you can only take 1 or 2 steps at a time.

The tower has N levels, and you are currently on the ground floor (level 0). Each step can take you up one or two levels, but no more. Your task is to find out how many distinct ways you can reach the top of the tower (level N).

Note: The old sorcerer has forbidden the use of arrays or strings to track your progress! You must rely on the pure magic of recursion to find the solution.

Input Format:

A single integer N , representing the number of levels in the tower.

Output

A single integer, the number of distinct ways to reach the top of the tower.

Q2: The Tower of Magic

Example

Input

3

Output

3

Explanation:

There are three ways to reach level 3:

1 -> 1 -> 1,

1 -> 2,

2 -> 1

Q3: Recursive Fast Multiplication

You are tasked with writing a C program that implements a recursive algorithm for fast multiplication of two large n -digit numbers. The method is based on dividing the numbers into smaller parts, recursively solving for their product, and recombining them to get the final result.

Explanation (Simplified):

- ❑ Suppose you have two large numbers x and y , each with n digits.
- ❑ To multiply x and y , you divide them into two smaller numbers.
- ❑ Split x into two parts: a (most significant half) and b (least significant half).
- ❑ Similarly, split y into c and d .
- ❑ You now have the four products: ac , bd , ad , and bc .
- ❑ Using these, the product of x and y can be recombined as:

$$(10^m \cdot a + b) \cdot (10^m \cdot c + d) = 10^{2m} \cdot ac + 10^m \cdot (ad + bc) + bd$$

- ❑ The recursion happens when you calculate the products ac , bd , ad , and bc recursively until you reach single-digit multiplication.
- ❑ Multiplying by powers of ten is simple: just shift the result by adding zeros to the end of the number.

Q3: Recursive Fast Multiplication...

Your Task:

Write a recursive C function that multiplies two large integers using this divide-and-conquer strategy. The input is two integers and the number of digits n in each integer.

Input Format:

The first line contains three integers: x , y , and n (the number of digits in both x and y).

Output:

The printf statement for the output is already given in the template. Please do not change it. Please do not include any statement in between the printf and the return statements. Please do not write any additional printf statements. The printf statement we gave is supposed to print the values of ac , bd , ad and bc ; and the resultant product for the current function call.

Example 1:

Input:

12 45 2

Output:

The ac , bd , ad and bc are : 4 10 5 8

Q3: Recursive Fast Multiplication...

Example 2:

Input:

324 567 3

Output:

The ac, bd, ad and bc are : 0 15 0 0. The partial product=15

The ac, bd, ad and bc are : 12 28 14 24. The partial product=1608

The ac, bd, ad and bc are : 0 21 0 18. The partial product=201

The ac, bd, ad and bc are : 0 20 10 0. The partial product=120

The ac, bd, ad and bc are : 15 1608 201 120. The partial product=183708

Product of 324 and 567 is: 183708